

面向多 Agent 的容器化沙箱 基础设施实践

演讲嘉宾：袁瑞杰

目录

CONTENT

01 基于容器构建的Agent Sandbox

02 Agent Sandbox关键技术实现

03 阿里云开源的OpenKruise Agents

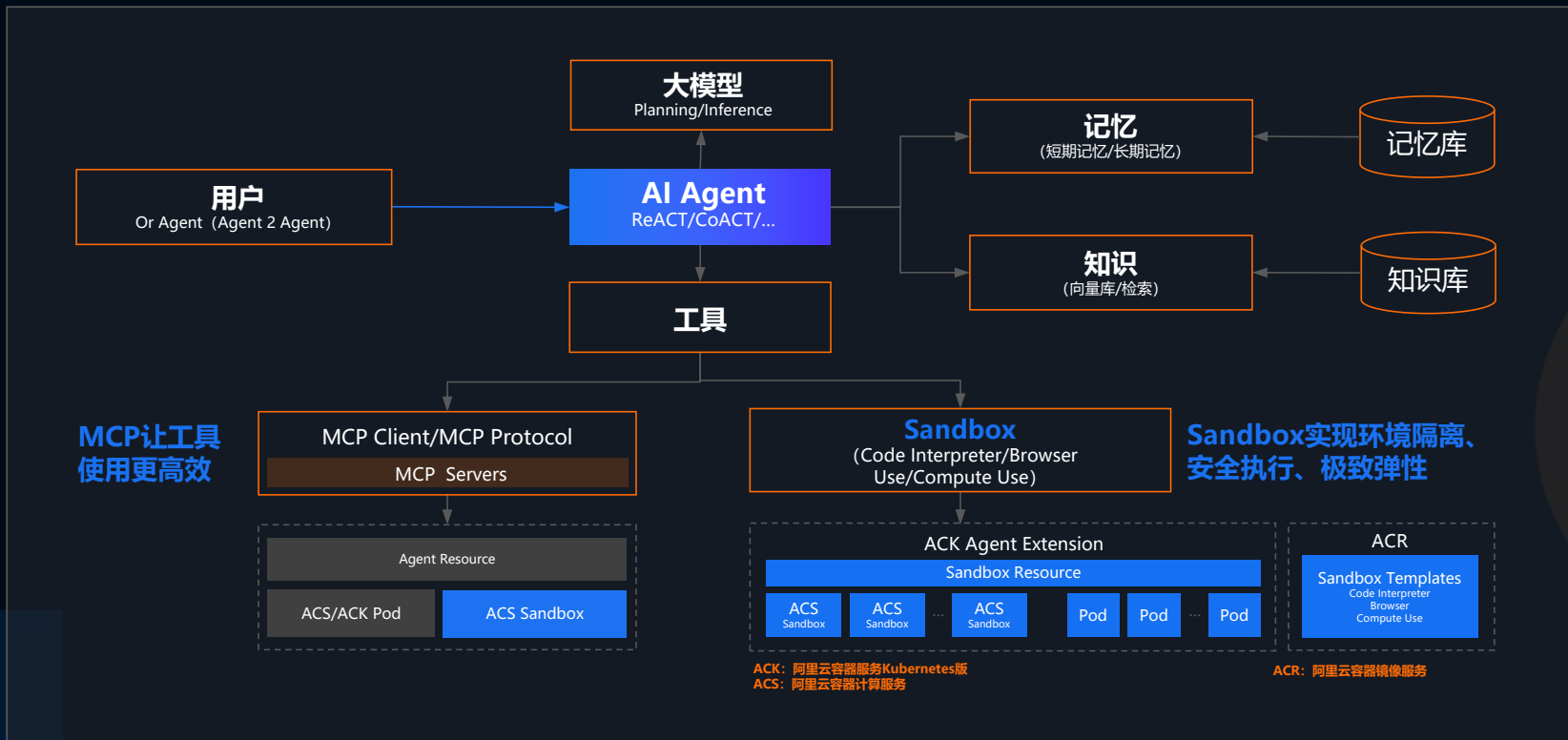
04 Agent Sandbox生态

05 客户案例

01

基于容器构建的Agent Sandbox

阿里云容器服务智能体方案简图



智能体应用落地面临的业务挑战

Sandbox: 高安全性、状态保持场景



数据安全

1. 攻击者提示词诱导恶意行为模型动态生成不可预期代码;
2. 多会话数据需要严格隔离

算力 需要运行在安全隔离的环境



大规模极速交付

1. 模型动态控制工具的执行, 存在更大规模的秒级交付弹性并发
2. 会话数量、会话并行度, 加剧资源需求动态波动

算力 要极致弹性能力



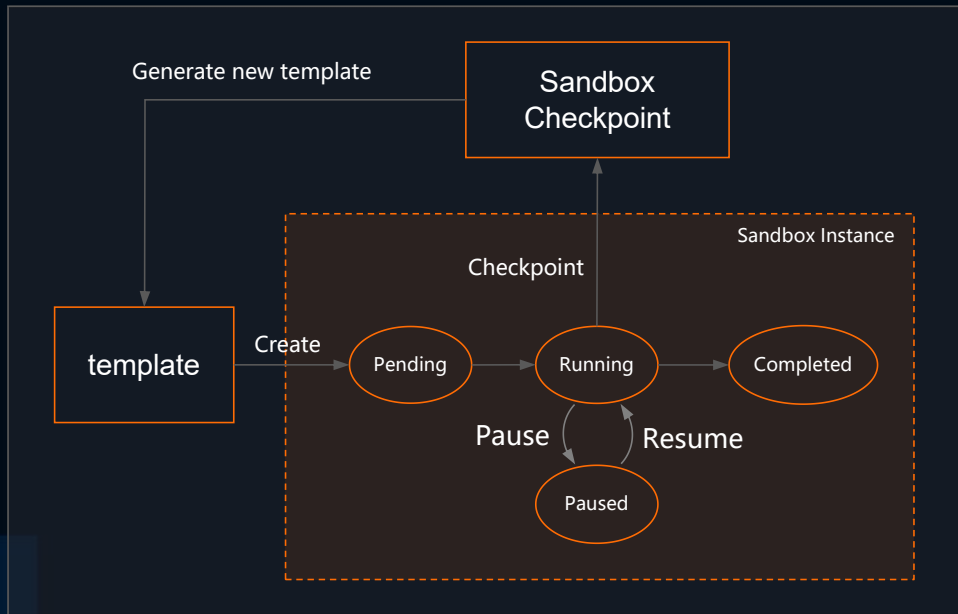
状态持久化

1. AI Agent 长周期/多步骤/强状态存在跨多轮交互与工具调用
2. 多会话并存加剧沙箱规模膨胀
3. 不是所有会话时刻存活

状态 要能有效保持
成本 要能合理控制

智能体应用落地面临的业务挑战

Sandbox: 高安全性、状态保持场景



复杂的沙箱生命周期

- 01** 模板: 包括镜像、编排和可选的 checkpoint
- 02** Paused: 资源占用最小化 (无CPU等消耗)
- 03** Checkpoint: 内存、临时存储和显存状态数据

02

Agent Sandbox关键技术实现

Sandbox安全隔离的典型技术实现

应对数据泄露、代码注入、网络攻击 等安全风险

计算隔离

- CPU/内存相互隔离, 互不干扰

网络隔离

- 禁用东西向网络
- 南北向单向连通
- 独立公网访问

存储隔离

- 共享存储的挂载点隔离

鉴权隔离

- 单Agent的独立rbac鉴权

可观测

- 所有Agent行为可追踪、可审计

大规模极致交付的容器资源管理技术

Sandbox资源管理的复杂度

- Sandbox运行时间短；对启动速度要求高
- Sandbox需等待人类或其它工具反馈，等待时间长，整体生命周期时长难预测
- Sandbox资源消耗难预测
- 极致弹性的技术实现复杂度高

Agent Sandbox 资源管理

Serverless算力

业界典型方案：

- 阿里云ACS Pod
- AWS Fargate
- Azure AKS Pod
- GKE Agent Sandbox

绝大部分Agent用户倾向优先使用
Serverless以简化Sandbox使用复杂度

K8S节点池

业界典型方案：

- 阿里云runD
- Kata/kata on PVM
- Firecracker
- gVisor

安全沙箱技术、二次虚拟化技术；计算、
存储、网络隔离等技术

算力层 物理算力底座

AMD EPYC

Intel Xeon

高核密度 单颗最高 192C/384T · 高整合比 约 7:1、TCO 最高降 67% —— 支撑海量沙箱高密度部署与低成本弹性

Sandbox状态保持技术的实现

文件系统的数据保持

容器rootfs数据，临时卷，持久化卷的数据检查点保留能力。

内存/显存数据保持

CRIU、VM Memory Checkpoint等技术实现内存数据的快照点保留能力；NVIDIA/cuda-checkpoint等显存保持能力。

两种开源方案的思路：

firecracker

文件系统使用数据持久化存储

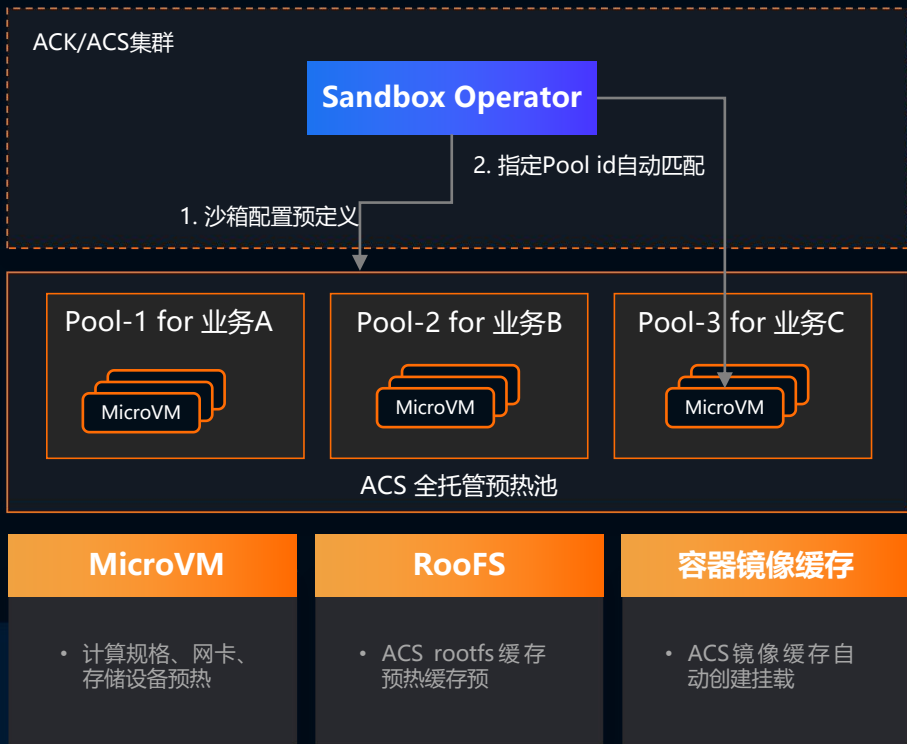
VM维度的内存Checkpoint

gvisor

文件系统使用数据持久化存储

gvisor的内存CRIU技术

阿里云ACS Sandbox极速唤醒工程实践



1. 池化技术：同构沙箱配置预定义，自动匹配实现极速启动与唤醒；预热池管理产品化

2. Serverless模式：具备大规模极致交付能力的Serverless算力，简化Sandbox资源管理

3. 基于已有的ACS安全沙箱隔离技术、网络、存储 隔离能力

4. 基于阿里云块存储的快照预热与复制能力，实现极速Rootfs恢复

5. Sandbox Operator简化客户使用方式，管理Sandbox生命周期；屏蔽预热池管理复杂度

• **Sandbox Operator**生产实践经验进一步开源到 **OpenKruise Agents**社区

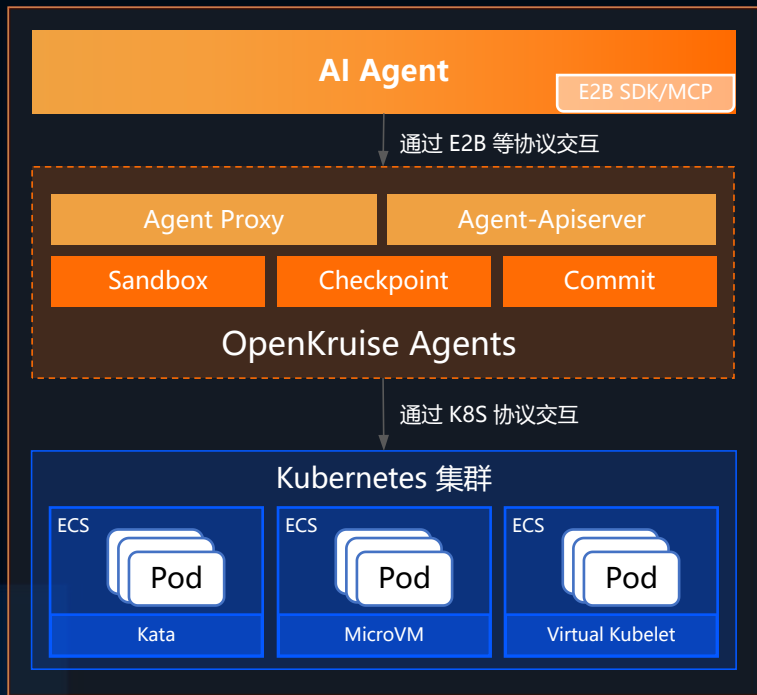
阿里云ACS Sandbox休眠唤醒运行时生产经验



03

阿里云开源的 OpenKruise Agents

应用生态层的衔接：阿里云开源的OpenKruise Agents



OpenKruise Agents提供了管理 Sandbox 生命周期的标准能力：

- Sandbox 创建、休眠、唤醒，包括内存、读写层数据等
- 高效资源供给：通过资源池化和资源变配等技术，加速沙箱的冷启动时间
- Checkpoint/Fork 能力，满足 RL 训练任务

同时，OpenKruise Agents 也是 AI Agent 与 Kubernetes 的中间纽带，对外提供 Agent 工具调用能力

- 向上通过E2B等协议，对接 AI Agent
- 向下通过K8S协议，使用 Pod 承载 Sandbox

OpenKruise Agents: AI Agent集成方式

Python SDK (E2B兼容)

面向 AI 科学家 / 开发者:

- Agent-Apiserver 支持通过 E2B等协议 的 REST/gRPC 接口, 无缝对接 E2B 原生 Python SDK
- 支持 create()、sleep()、wake()、destroy() 等高级生命周期操作
- 无需了解 Kubernetes 细节, 以代码原生方式管理 Sandbox 实例, 快速嵌入 AI Agent workflow

```
# Import the E2B SDK
from e2b_code_interpreter import Sandbox

# 指定Sandbox Template, 创建 Sandbox
sbx = Sandbox.create(template="code-interpreter-custom", timeout=300)
print(f"sandbox id: {sbx.sandbox_id}")

# 通过 Sandbox RunCode
def execute_python_code(s: Sandbox, code: str):
    # Execute Python code inside the sandbox
# runcode 例子
execute_python_code(sbx, "print('hello world')")
```

Sandbox CR

面向 Agent 平台/运维工程师:

- 根据 Sandbox CR 自动完成 Pod 创建、状态同步、休眠调度与资源回收
- 类似 Deployment, 通过 CRD SandboxSet 声明期望的 Sandbox 实例数量与配置

```
apiVersion: agents.x-k8s.io/v1alpha1
kind: Sandbox
metadata:
  name: my-sandbox
spec:
  podTemplate:
    spec:
      containers:
      - name: my-container
        image: <IMAGE>
```

OpenKruise Agents 项目整体说明

- Agent-Sandbox 是 CNCF 孵化项目 OpenKruise 社区开源面向 AI Agent Sandbox 的端到端解决方案，旨在提供标准化的 Sandbox 生命周期管理能力。

Agent Apiserver

支持社区E2B等API

Sandbox路由管理

Sandbox管理

Sandbox 休眠/唤醒

Sandbox 池化

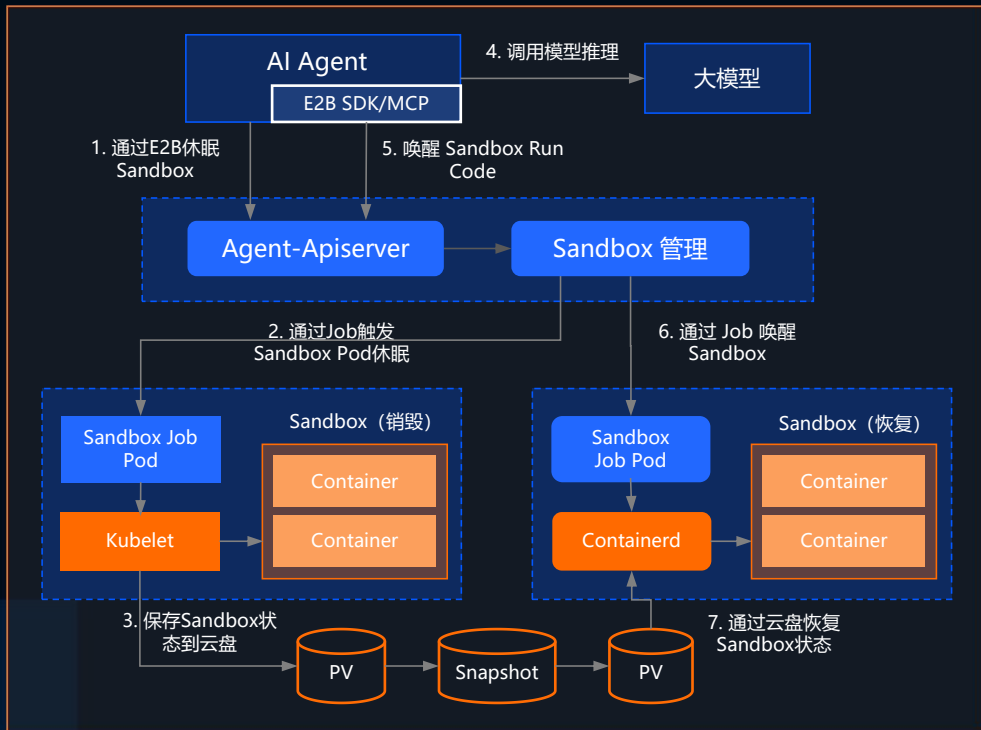
Checkpoint管理

Pod的快照/克隆

容器的镜像Commit

项目地址: <https://github.com/openkruise/agents>

OpenKruise Agents : Sandbox 实例状态休眠、唤醒

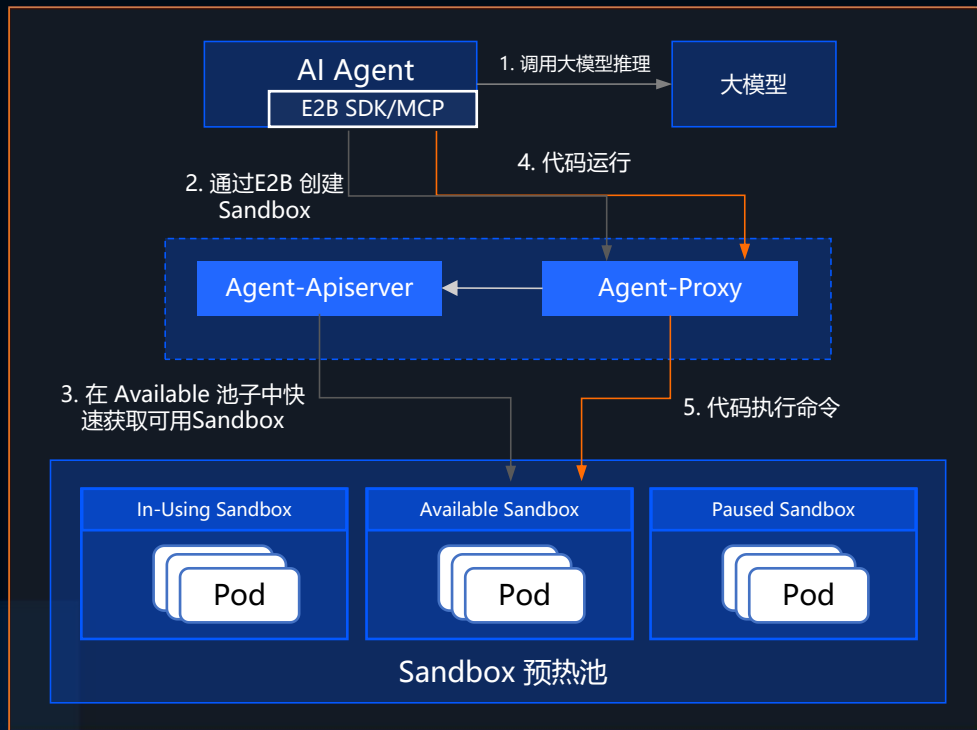


Sandbox 实例状态生命周期管理:

- **休眠 (Sleep)** : 空闲时, 可将其转入休眠状态, 降低成本
- **唤醒 (Wake)** : 当 AI Agent 再次需要执行任务时, 能快速恢复 Sandbox 到休眠前的状态

```
apiVersion: agents.kruise.io/v1alpha1
kind: Sandbox
metadata:
  name: sample
spec:
  pause: false
  persistentContents:
    - memory
    - filesystem
    - ip
    - gpuMemory
  template:
    metadata:
      labels:
        agent: sample
  spec:
    containers:
      - name: my-session
        image: session:v1
```

OpenKruise Agents : Sandbox 池化扩容



Sandbox 池化扩容:

- **快速获取**: 预热池中秒级分配就绪实例，避免冷启动延迟
- **快速路由**: 通过内置的Agent-Proxy (Envoy、Proxy-Manager) 边车对Sandbox进行路由

```
apiVersion: agents.kruise.io/v1alpha1
kind: SandboxSet
metadata:
  name: demo
spec:
  replicas: 10
  template:
    metadata:
      labels:
        agent: testpause001
    spec:
      containers:
        - image: nginx:1.14.1-8.6
```

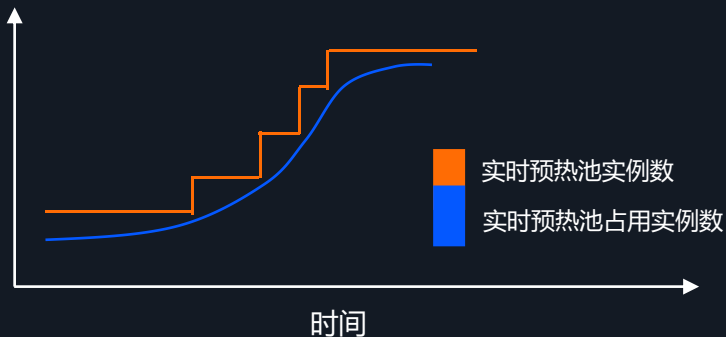
OpenKruise Agents : Sandbox 池化扩容

降低预热成本

AutoScaler

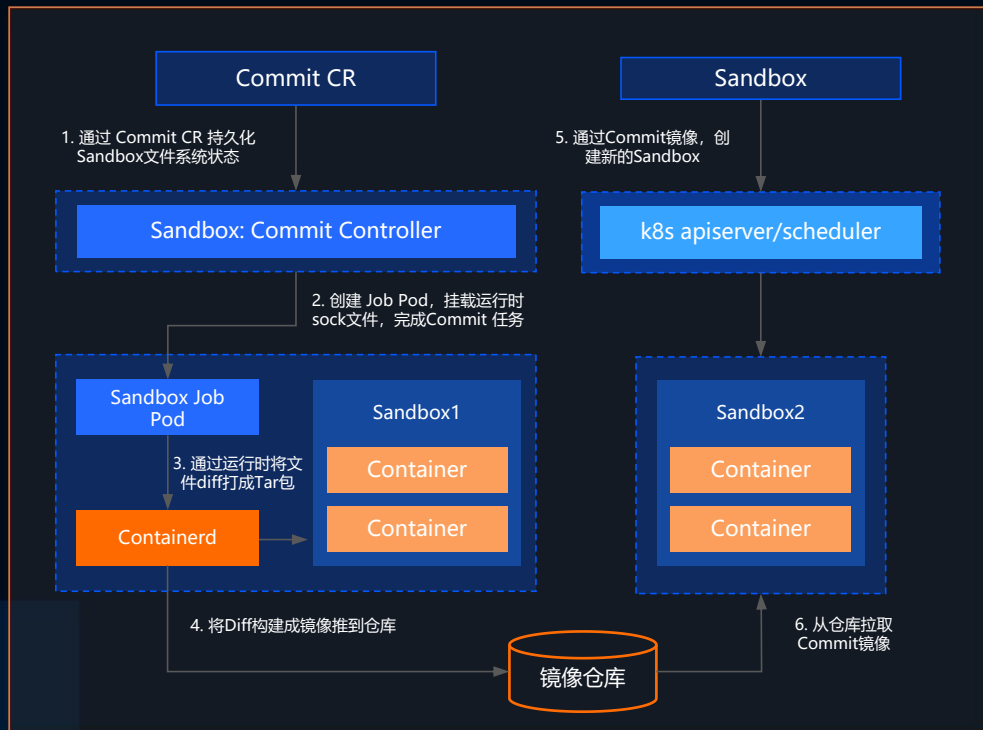
Refresher

- 基于预热池水位的弹性
- 基于时间的弹性
- 已使用的Sandbox复用
- Sandbox镜像的动态更新



```
apiVersion: agents.kruise.io/v1alpha1
kind: PoolingAutoScaler
metadata:
  name: code-autoscaler
spec:
  scaleTargetRef:
    apiVersion: agents.kruise.io/v1alpha1
    kind: sandboxset
    name: code-interpreter
  observeWindowSeconds: 600
  # 预热池大小限制
  minReplicas: 3
  maxReplicas: 100
  # 基于预热池水位的弹性
  minAvailableRatio: 20%
  maxAvailableRatio: 90%
  # 基于时间调度的弹性
  schedule:
    - startTime: "2025-12-1 00:00:00"
      endTime: "2031-12-12 00:00:00"
      bounds:
        - cron: "* 0-8 ? * MON-FRI"
          minReplicas: 10
          maxReplicas: 150
```

OpenKruise Agents : 状态保持 Commit

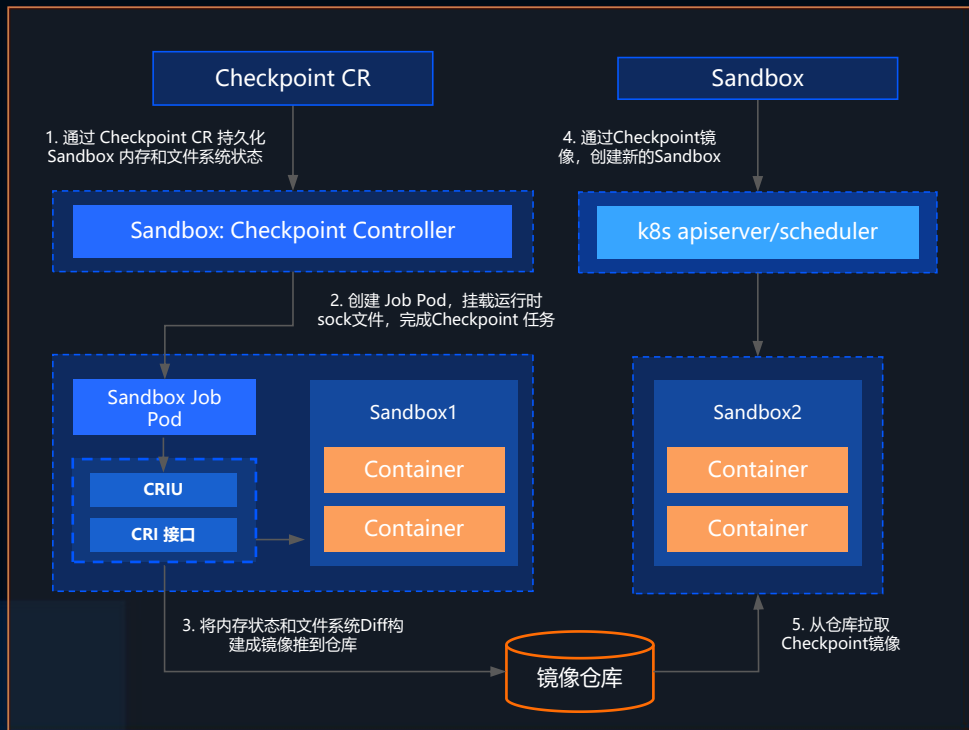


Commit CR: 持久化文件系统读写层

- 通过 CRI 接口, 捕获 Sandbox 的 overlays/rw-layer
- 将运行时文件系统差异层打包为标准 OCI 镜像
- 实现 Sandbox 状态的可回溯、可重建、可迁移

```
apiVersion: agents.kruise.io/v1alpha1
kind: Commit
metadata:
  name: commit-01
spec:
  # commit Pod对象
  podName: commit-01
  # commit 容器名
  containerName: main
  # commit 镜像地址
  image: commit/nginx:commit-01
  # 回收 Commit CR 策略
  ttl: 72h
  # push 镜像鉴权
  registryAuth:
    secrets:
      - push-secrets
```

OpenKruise Agents : 状态保持 Checkpoint



Checkpoint CR: 持久化内存和文件系统读写层

- **文件系统层**: 调用 CRI 接口, 导出容器 rw-layer 为标准 OCI 镜像
- **内存与进程状态**: 通过 CRIU 执行 checkpoint, 保存寄存器、内存页、网络连接等上下文
- **输出**: 一个可重建的完整 Sandbox 快照 (镜像 + 内存 dump)
- 当前通过镜像commit实现, 未来通过云盘实现

```
apiVersion: agents.kruise.io/v1alpha1
kind: Checkpoint
metadata:
  name: checkpoint-demo-1
  namespace: default
spec:
  # 目标Pod Name
  podName: pod-demo-1
  # checkpoint之后是否要求Pod处于Running状态。
  # 如果是 false, Pod状态变为 Succeeded
  keepPodRunning: false
  # Checkpoint 回收时间, ttlAfterFinished 后会主动回收 checkpoint 资源
  ttlAfterFinished: 30m, 30h, 30d
  persistentContents: # 目前仅支持 memory,filesystem; filesystem 两种组合。默认启用 memory,filesystem
    - memory
    - filesystem
```

04

Agent Sandbox生态

Openkruise Agents: 与Agent其它生态的集成

- Multi-Agent Collaboration
- Workflow
- LLM abstract & Reasoning

Agent Framework

LangGraph

AgentScope

Kagent

Dapr Agent

复杂工具抽象

- Resource provision
- State Persistent

Sandbox Infra

OpenKruise Agents

AgentCube

SIG Agent-Sandbox

- Tool integration
- OS Emulation

Agent Runtime

AIO Sandbox

redroid

AndroidWorld

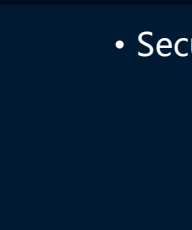
功能能力实现和封装

- Secure Fast Runtime

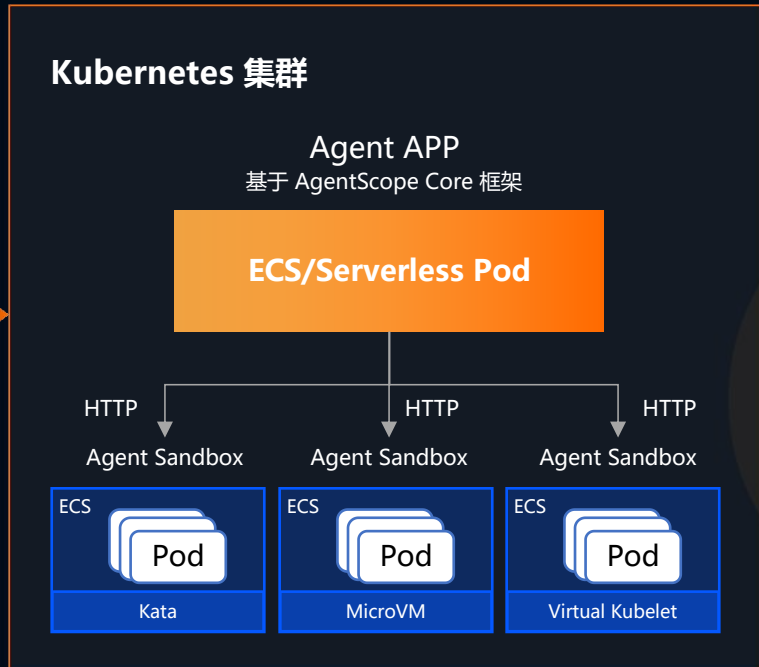
Container & Virtualization

Kata

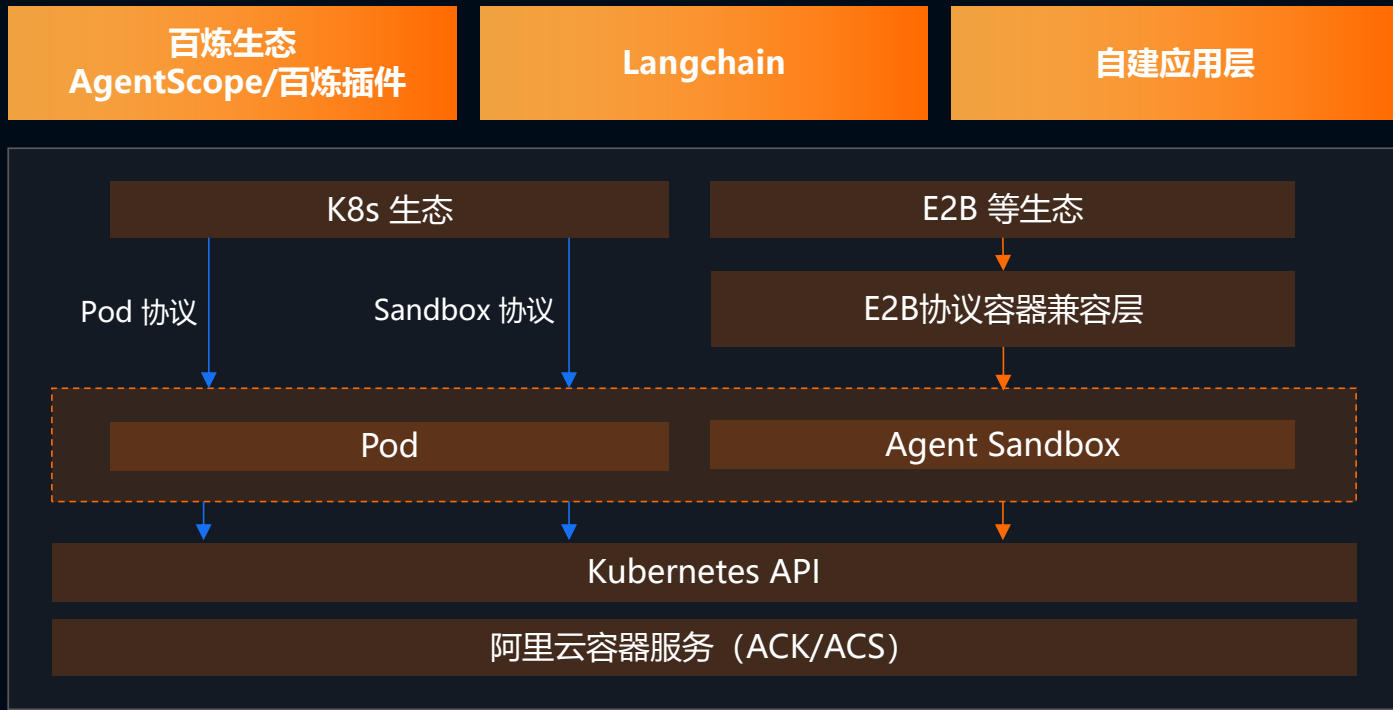
gvisor



AgentScope on K8s



智能体应用on K8s整体概览图



05

客户案例

客户案例：MiniMax 基于 ACK + ACS 重构 Agent 运行底座

MaxClaw / MaxHermes —— 控制平面与执行平面分离，海量 Agent 从「可用」走向企业级「落地」



客户案例：Kimi 基于 ACK + ACS 的Agent Infra

深度研究 / OK Computer 等 C 端 Agent + K2 模型 RL 训练 —— 常态算力与 Serverless 分级调度，兼顾性能、成本与稳定

业务场景 & 痛点

- **C 端高峰数万并发**
每个请求秒级要独立算力，容器分钟级冷启动不可接受
- **执行大模型生成代码**
未经人工验证，多租户下必须硬隔离
- **研究型长任务易断线**
中间数据 / 推理路径一旦销毁，用户被迫从头再来
- **RL 训练批量启停**
峰值预置资源浪费巨大，需按需弹性压成本

ACK ResourcePolicy 分级调度

排队 > 500 / 等待 > 30s
→ 溢出

① ACK 节点池 · 常态基线

多可用区 / 多规格即时弹性 · 自定义镜像 + 数据盘快照 · Terway ENI 预分配 → 千节点分钟级扩容

② ACS Agent Sandbox · Serverless 溢出

MicroVM 轻量虚拟化，秒级拉起数千沙箱 · Quota 热更新 burst · 休眠唤醒 · 内存级 Checkpoint 克隆

90%

虚拟化开销降低

MicroVM 轻量虚拟化，
数千沙箱秒级并发启动

60%+

沙箱启动提速

镜像缓存 + Quota 热更新 burst，
Python 类沙箱启动时间缩短过半

10万+

Pod 规模稳定调度

调度器性能达开源数倍，
千节点每秒数百 Pod 稳定运行

数秒克隆

RL 分支路径探索

内存级 Checkpoint 快照，
瞬时克隆数千副本供 MCTS 探索

THANKS

演讲嘉宾：袁瑞杰